openAG — A Graph-Native Architecture Engine (Design)

Status: Design only (brainstorming). Implementation plan deferred. Date: 2026-10-04 (rev. 2026-10-12 — neural govern, VoI economics, B0, risk, integration/right-of-Vee, SE-completeness audit added; §8.2 and §3.4 superseded). Author: Modeler + OpenCode (brainstorming session) Relationship to AMS: openAG is a greenfield sibling to architecture-model-standard (AMS). AMS remains frozen as the production extractor until openAG reaches parity.

> 2026-10-12 revision ledger (this session). §3.4 ground now mixed per-axis + four fact-source oracles; §8.2 `BudgetGovernor` replaced by neural govern (confluence-safe accelerator); §8.3 VoI economic model (budget = value-of-information allocator; risk collapses in); §8.4 gate two layers (well-formedness + go/no-go decision authority); §2.1 Extract = B-ROUND to fixpoint; §2.2 conservation laws; §15 B0 stakeholder-needs; §16 integration/right-of-Vee; §17 SE-completeness audit (trade-space + lifecycle = diachronic meta-ROUND); §18 attribute four end-states + reservoir; §10 gap register extended (`gap:trivial`, `gap:low-confidence`, `gap:integration-mismatch`).

---

1. Motivation

AMS today is a linear batch pipeline (`observe → infer → allocate → relate → specify → contract → validate → decompose → synthesize → emit`) that produces three separate representations (the YAML model, the reality manifest, and the derived views/artifacts) which must then be kept in sync. This creates three structural problems:

1. Representation drift — model, manifest, and views are separate artifacts reconciled by convention, not construction.
2. Batch rigidity — the pipeline runs end-to-end; there is no single incremental operator you can re-run on one node.
3. Opaque growth — "why does this entity exist?" is answered by pipeline-stage archaeology, not by a first-class record.

openAG replaces all of this with one mutable **`ModelGraph`** grown by a single promise-discharge mechanism. There is one representation (the graph), one operator (discharge an open promise into evidence-grounded edges/nodes), and one honesty rule (emit a gap when you cannot).

> Tagline: a self-maintaining symbolic substrate that a neural half proposes into, verifies against executable evidence, and improves from.

Scope honesty

openAG is a structural modeling language for decomposable engineered systems — not a universal semantics engine. System dynamics are not modeled directly; they are delegated to A7 evidence (typed, optionally executable). Creation/regeneration completeness holds where a verification oracle is obtainable, not unconditionally. The system is proprioceptive (self-model + drift sensing) and neurosymbolic, but not sentient — its self-reference is reframed as homeostatic self-maintenance.

---

2. Core principle

> A system model is the least fixpoint of promise-discharge: from seed nodes, repeatedly discharge each node's open promises — along whichever decomposition axis they point — into evidence-grounded edges (and the nodes they require), normalizing to canonical form each round, until no open promise remains within budget. Shape and direction are emergent from kind-valence; honesty is preserved by emitting gaps.

The graph's topology is never authored directly. It is the residue of discharging promises. Every node carries open promises dictated by its kind's valence (§3, §4); discharging a promise either grounds an edge (and possibly a required node) in evidence, or — when no grounding is found within budget — emits an honest gap.

2.1 Extract is B-ROUND run to promise-fixpoint

Extract is not a bottom-up one-shot pipeline. It is B-ROUND (§3.3) run until the promise-fixpoint, with direction top-down intent, bottom-up grounding. The old linear `pipeline_v3` bottom-up `scan` survives only as a fact source feeding **`ground`** (the code-scan oracle, §3.4) — it no longer drives structure.

From iteration 0, every entity carries open promises on all valent axes (there is no privileged axis). `govern` schedules whatever is ready + affordable; A1 `contains` decomposition appears early only as an emergent effect of the promise-dependency DAG, never as a hardcoded phase. Branching is unbounded-propose, ground-filters, govern/budget- caps. Downward termination is a structural fixpoint per branch (a branch is atomic when it has no cohesive sub-structure or hits a single-module floor; the old ≥5-file decomposition rule degrades to a grounding signal, not a gate).

2.2 Conservation laws

Two axes carry conservation constraints that make the graph well-formed:

- A1 **`contains`** = strict partition. Decomposition is a partition; the module is the partition atom. Cross-cutting concerns that would violate the partition escape via a shared component or `LAY-CrossCutting`, never by double-containment.
- A2 **`realizes`** = many-to-many, free. Allocation is unconstrained; one capability may be realized by many components and vice-versa.

Escape hatches preserving honesty: a residual `gap:undecomposed` blob, shared components, and gap-honest termination.

2.3 Re-binding: fluid then sticky

Identity is fluid until grounded, then sticky. `reconcile` owns re-binding via the σ θ-crosswalk thresholds (§8.1): `σ ≥ 0.85` bind / `0.60–0.85` ambiguous / `< 0.60` new. One operator — **`reconcile`** + σ — is the single binding mechanism for modules, attributes, AND cross- generation identity.

---

3. Formal core

3.1 The 7 decomposition axes

Promises point along one of seven axes. Together these span AMS's 23 relationship types (the real `RelationType` enum in `core/types.py` / `schema.json`, not the 17 originally quoted) plus the two synthetic graph-export edge kinds (`has-promise`, `verified-by`).

Axis	Name	Relation types	Direction
A1	Abstraction	`contains`	↓ (expand) / ↑ (roll-up)
A2	Realization	`realizes`, `allocated-to`, `satisfies`	↔

Axis	Name	Relation types	Direction
A3	Boundary	`exposes`, `consumes`, `depends-on`, `uses`, `mounted-on`, `connected-at`	↔
A4	Governance	`constrained-by`, `affects`, layer membership	↕
A5	Flow	`triggers`, `produces`, `traces-to`, `transforms`, `subscribes-to`, `routed-through`, `derives-from`	→
A6	Lifecycle	`supersedes`, `migrates-to`, `resolves`	→ (v2-deferred)
A7	Evidence	`verified-by`, `verifies`, `has-promise`	⊥ (universal)

All 23 canonical relation types are covered: `realizes`, `contains`, `depends-on`, `exposes`, `consumes`, `traces-to`, `allocated-to`, `constrained-by`, `mounted-on`, `connected-at`, `routed-through`, `produces`, `subscribes-to`, `transforms`, `supersedes`, `migrates-to`, `triggers`, `satisfies`, `derives-from`, `verifies`, `resolves`, `affects`, `uses` → mapped above. `has-promise` / `verified-by` are the synthetic export edges.

A6 has no expander in v1 (deferred), and this is sound because no v1 kind owes a promise on A6 (§3.5 total-registry invariant).

Ownership is A1↑. Hierarchical ownership of any I/O entity (artifact, journal, WorkOrder, …) is the transitive `contains` (A1-abstraction, roll-up direction) chain of its producing entity up to the System. See the entity-model doc §I/O-entity family.

3.2 The one-step operator

A single promise is discharged by six sub-stages — identical to the six `model.*` sub-capabilities, 1:1:

`select → propose → ground → admit → reconcile → bind`

- select — BudgetGovernor picks the next open promise (§8.2).
- propose — an expander (symbolic or neural) proposes candidate edges/nodes to discharge the promise.
- ground — candidates are checked against evidence (A7); ungrounded candidates are rejected or downgraded to a gap.
- admit — the GateEvaluator admits only candidates that satisfy constraints and schema.
- reconcile — the Reconciler normalizes to canonical form (dedup, identity-bind, roll-up aggregation). This is a closure operator.
- bind — IdentityStore binds stable IDs via the sticky θ-crosswalk (§8.1); GenerationStore records the new generation.

3.3 The round and the fixpoint

A round of `B-ROUND` is: `govern → grow → ground → reconcile → bind → gate`, triggered by `B1 Extract`, `B2 Author`, `B3 Re-extract`, or `B6 Learn`. Rounds repeat until no open promise remains within

budget.

This is a monotone worklist / dataflow framework:

- expanders are transfer functions,
- `reconcile` is the join (a closure operator),
- the budget is the widening operator that guarantees termination.

Because `reconcile` is a closure operator, the run is confluent modulo budget: scheduling order (§8.2) changes which gaps appear under a budget cut, never the canonical grounded form.

3.4 The neurosymbolic seam

Stage	Half
propose (grow)	NEURAL
ground	MIXED (per-axis)
select (govern)	MIXED — symbolic legality/budget + neural prioritization
admit (gate)	SYMBOLIC
reconcile	SYMBOLIC
bind	SYMBOLIC

> Neural proposes, symbolic guarantees. The neural half never writes to the graph directly; every neural proposal passes through symbolic admit/reconcile/bind.

Ground is mixed per-axis (supersedes the pure-neural marking above). Grounding is split by the nature of the evidence, not done wholesale by one half:

- Structural promises → symbolic grounding. Anything checkable against a deterministic fact source (code-scan AST, declared schema, machine- evaluable constraint predicate, static interface consistency) is ground by the symbolic half. Deterministic ⇒ confluent.
- Semantic promises → neural grounding. Intent, rationale, naming, success-criteria satisfaction, "does this actually mean X" are ground by the neural half against the available oracle.

Govern is mixed, not purely symbolic. The legality filter and budget read are symbolic; the prioritization of the legal, affordable promise set is neural (§8.2, rewritten). This is safe because confluence (invariant #3) makes scheduling order correctness-neutral — govern is an accelerator, not a guarantor (invariant #6).

The four fact-source oracles. Grounding draws evidence from four interchangeable oracles, all arbitrated by govern's VoI (§8.3):

1. code-scan (static AST / artifacts) — cheap, deterministic.
2. human-dialogue — most expensive; used first when little else is grounded, fades as cheaper oracles become available (§B0).
3. model-self (the graph's own grounded state) — for roll-up / flow-down consistency.
4. runtime / composition-execution — right-of-Vee; grounds the dynamic promises of interfaces and behaviors (§Integration).

3.5 The 6 invariants

1. Total-registry — every `(kind, axis)` pair that the valence table marks "required" has a registered expander (or the axis is deferred and no kind owes it). No promise can be un-dischargeable by construction.
2. Grounding-bounded termination — every discharge either grounds in finite evidence or emits a gap; the budget widening guarantees the worklist empties.

3. Monotone lattice — the graph only grows or narrows gaps within a round; reconcile is monotone on the information lattice.
4. Identity-binding — IDs are sticky across generations via the θ-crosswalk (§8.1); no silent re-binding.
5. Gap-honesty — anything not grounded within budget is emitted as a typed gap, never silently omitted or hallucinated.
6. Neural-never-replaces-fixpoint — the neural half may accelerate or propose, but the symbolic fixpoint is always the source of truth.

---

4. Attribute algebra

Attributes have a lifecycle: absent → promised → filled → discharged. An attribute becomes a promise when the kind's valence requires it and it is unfilled. There are three loci — and zero new components are needed to support them:

> Anchoring note. An openAG promise is not a new type — it is the existing v3 `PromiseSpec` (`attribute, content, owner_entity, delivered_from, verified_by, status, gap_type, provenance`; schema `promise-graph.schema.json`, `PromiseStatus ∈ {open, delivered, verified, typed-gap}`) extended with two net-new fields: **`axis`** (which decomposition axis the promise points along) and **`valence`** (the kind-rule that demanded it). Everything else — content, evidence refs, gap typing, delivery chain — is inherited unchanged, so all prior v3 effort is preserved. The lifecycle `absent → promised → filled → discharged` maps onto v3 `(no promise) → open → delivered → verified`, with `typed-gap` as the honest-failure terminal.

1. node-local — the attribute lives on the node; discharged by an expander filling it from evidence.
2. roll-up (A1↑) — aggregated from descendants, computed in `reconcile`, with honest gap-inheritance (a parent's roll-up carries its children's gaps).
3. flow-down (A1↓) — a parent emits child attribute-promises verified by a parametric constraint (e.g. a latency SLO decomposed across a behavior chain). This is also the seam to delegated physics: a parametric behavior node's budget flows down and is checked by executable evidence.

4.1 Per-kind valence table

Base row (every kind): `id, name, kind, status, intent`, plus ≥1 A7 grounding or an explicit `gap:ungrounded`.

Kind	Required attribute-promises (absence ⇒ gap)	Roll-up (A1↑, in reconcile)	Flow-down (A1↓)
System (L0)	mission/F0, success_criteria, top-constraints	health ← Σ capability health	budget → capability budgets
Capability (L1)	intent, ≥1 `realizes` (else `gap:unrealized`), ≥1 sub-cap or leaf-justification, owner, success_criteria	realization-coverage ← children	SLO → sub-cap SLOs
Sub-capability (L2)	intent, parent (A1↑), ≥1 realizing component, verification	—	—

Kind	Required attribute-promises (absence ⇒ gap)	Roll-up (A1↑, in reconcile)	Flow-down (A1↓)
Component	layer (A4), ≥1 `realizes`-back (else `gap:orphan`), interfaces (A3), dependencies_rationale, anchor	evidence-coverage ← grounded/total	latency/token budget split
Behavior	intent, ≥1 `traces-to` (A5/A2), trigger (A5), steps, success_criteria, failure_modes; parametric: latency/throughput budget + executable-evidence	gap-count ← descendants	budget across behavior chain
Interface	subkind, ≥1 exposer + ≥1 consumer (A3, else `gap:dangling`), schema/contract, verification	—	—
Constraint	verification class, scope (A4), machine-evaluable predicate (C-N2, else `gap:unevaluable`)	—	—
Actor	type, ≥1 consumes/triggers (A3/A5, else `gap:disconnected`)	—	—
Layer	≥1 contained component (A1), position (A4), allowed-dep-direction	—	—
Evidence (A7 node)	uri, type (static/observational/executable), digest, verifies-target	—	—
Promise (node)	attribute, owner, axis, status, valence-rule-ref	—	—

5. Entity model (the locked ledger)

SE discipline: functional decomposition (capabilities → sub-capabilities) first, then allocate functions to components.

L0 — System

openAG — F0: “Sustain a verified architecture graph of a target system.”

L1 — 5 Capabilities

`extract · model · represent · learn · mdd`

L2 — 18 Sub-capabilities

- extract (3): scan · detect · seed
- model (7): grow · ground · gate · reconcile · govern · remember · traverse
- represent (3): slice · project · render
- learn (3): observe · lesson · tune
- mdd (2): generate · verify-parity

Depth is non-uniform (SE-correct). Decomposition continues per branch until a function is atomic and allocatable to one component — not to a fixed level. Atomic leaves (`scan`-leaf, `render`, `tune`, `seed`) stop at L2; non-atomic branches (`model.*`, `govern`, `reconcile`, `ground`, `slice`, `generate`) decompose to L3+ — those L3 functions are the operator stages (§3.2) and the §8 algorithms, promoted into the functional hierarchy. Full L3 enumeration lives in the entity-model companion doc. Uniform-depth decomposition would violate both classical functional analysis (INCOSE/NASA) and openAG's own emergent-depth principle.

26 Components (allocated to sub-capabilities)

- extract (3): `SourceScanner` · `PatternDetector` · `Seeder`
- model (12): `ExpansionKernel` (coordinator) · `PromiseEngine` · `ExpanderRegistry` · `Grounder` · `GateEvaluator` · `Reconciler` · `BudgetGovernor` · `ModelGraph` · `DecisionCache` · `IdentityStore` · `GenerationStore` · `WalkEngine`
- represent (4): `Slicer` · `ProjectorRegistry` · `Renderers` · `Viewer`
- learn (3): `FeedbackJournals` · `LessonStore` · `RuleTuner`
- mdd (2): `CodeGenerator` · `ParityChecker`
- support (2): `LLMProvider/Router` · `SeedStrategy`

~33 Behaviors

- B-ROUND — the shared fixpoint `govern → grow → ground → reconcile → bind → gate`, triggered by B1/B2/B3/B6.
- B1 Extract · B2 Author · B3 Re-extract · B4 Represent · B5 Generate · B6 Learn · B7 Gate (= B-ROUND's gate step standalone). Each top-level behavior fans out into sub-behaviors (one per model.* stage), ≈33 total.

~13 Interfaces

`ExpanderRegistry-API` · `ProjectorRegistry-API` · `LLMProvider-Protocol` · `Patch/Proposal-schema` · `ScanReport/Manifest` · `EvidenceRef (typed)` · `PromiseSpec/valence-table` · `WalkSpec` · `SliceSpec` · `ViewSpec` · `Journal-append-API` · `nn-compilable-graph-export` · `CLI/MCP-surface`.

Honest internal-only (no interface): `BudgetGovernor`, `DecisionCache`, `IdentityStore`, `SeedStrategy`, `LessonStore`.

5 Actors

`Modeler (human)` · `Target System (sensed reality — source of all A7 evidence)` · `LLM service (external)` · `Auditor` · `Downstream / arch-agent`.

4 Layers + 1 cross-cutting

`Substrate` (4: Scanner, Detector, Seeder, Journals) → `Kernel` (13) → `Projection` (6) → `Adaptation` (2: LessonStore, RuleTuner). Cross-cutting: `LLMProvider/Router`.

Constraints

Three verification classes drive the gap taxonomy: - existence → `gap:missing` - prohibition → `gap:violation` - parametric / budget → aggregate-verified (fitness functions)

Plus `ModelGraph-pure-state` (anti-god-object: the graph is pure state, all logic lives in operators) and C-N2 (NN-compilability, §6).

---

6. Neurosymbolic seam: N1 vs N2

Two distinct neural ideas, not to be conflated:

- N1 — GNN surrogate of the modeling process. Learns to imitate the openAG operator to speed extraction. Lives in arch-agent. Covered by the `propose`/`ground` neural half (§3.4). Not built here.
- N2 — compile the modeled system's graph into a differentiable network $\mathcal{N}(G)$. Each axis has a precise neural-execution semantics:

Axis	Neural-execution semantics
A1 Abstraction	module nesting
A2 Realization	weight-tying
A3 Boundary	data-flow wiring
A5 Flow	recurrence
A4 Governance	loss / regularizer
A7 Evidence	supervision

  Symbolic topology + neural parameters; constraints become losses, evidence becomes supervision. N2's compiler/trainer/distiller and the `CAP:mdd` neural backend are deferred to arch-agent.

But openAG must GUARANTEE every modeled system's graph is NN-compilable — expressed today as constraint C-N2, which forces three things into v1:

1. **`axis`** is a first-class, stored relationship field (not inferred).
2. A lossless graph-export acceptance test (every edge/node/attr round-trips through the `nn-compilable-graph-export` interface).
3. Node feature fields + typed evidence + machine-evaluable constraints (already chosen in §4.1 / §7).

---

7. Critique absorbed + 4 solutions

External critique reframed three overclaims (see §1 scope honesty). Four concrete solutions keep the ambition while staying honest:

1. Cyber-physical reach — parametric behavior nodes + typed/ executable evidence (A7 taxonomy: static / observational / executable). Executable evidence makes behaviors co-simulatable.
2. Creation completeness — move the precondition from "pre-pinned spec" to "oracle-obtainable": `B5` `Generate` runs a closed-loop behavioral-promise discharge with the oracle in the loop.
3. Homeostatic self-repair — self-model + proprioception + drift-detection + `B6` → `RuleTuner` → `B-ROUND` re-entry. Metric: drift-closure latency.

4. Reconcile-as-labeler — reconcile's accept/reject decisions auto-label the neural half. Metric: neural-acceptance-rate, which becomes a training reward for arch-agent's N1.

---

8. Resolved design decisions

8.1 Identity θ-crosswalk

Kind-gated, anchor-weighted similarity with a hysteresis band and bipartite stable matching.

- σ(n_old, n_new) ∈ [0,1], hard-gated on `kind` (mismatch ⇒ σ=0), then: anchor match 0.50, name similarity 0.25, structural context 0.25.
- σ ≥ θ_high = 0.85 → bind (reuse prior ID).
- 0.60 ≤ σ < 0.85 → ambiguous → fresh ID + `gap:ambiguous-identity`
  - deferred candidate-merge promise.
- σ < 0.60 → new entity.
- Hysteresis / stickiness: a bound identity persists across generations even if σ later dips, unless its anchor is destroyed (then it becomes a `gap:orphan` candidate — never silently re-bound).
- Bijectivity: within `(kind, parent)` scope, Hungarian stable matching prevents two new nodes claiming one old ID; the prior generation's crosswalk seeds the match.

8.2 Govern — neural prioritization in a thin symbolic box

Supersedes the deterministic **`BudgetGovernor`** above. The scheduler is neural, wrapped in a thin symbolic shell. This is safe precisely because confluence (invariant #3) makes scheduling correctness-neutral — govern becomes an accelerator, not a guarantor (invariant #6). Four steps:

1. Symbolic legality filter. Admit only promises whose dependencies are satisfied in the down-only, acyclic promise-DAG (re-read each round as the DAG grows). Up-flows are not scheduled here (see the compatibility principle below) — this keeps the DAG acyclic.
2. Symbolic budget read. Read remaining budget; set mode (expansion vs consolidation). Budget owns widening/termination (invariant #2): near exhaustion, stop proposing new nodes; discharge only gap-closing promises.
3. Neural prioritization. An LLM orders the legal, affordable set by semantic value (VoI, §8.3). This is the only neural step in govern.
4. Symbolic bundle + budget-clamp. Bundle same-`(task_class, tier, template)` promises (fork AMS Part-14 scheduler); clamp to budget. Free T1 symbolic work auto-drains first — so "T1-before-LLM" is an emergent effect of zero cost, not a hardcoded rule. Overflow emits `gap:budget-exhausted`.

Honest caveat. Under a budget cut, neural prioritization makes the partial-Generation gap-set non-deterministic (the grounded content stays confluent; only which gaps surface varies). Pin `temperature=0` for reproducible runs.

Compatibility principle (dissolves the flow-down/roll-up cycle). `govern` schedules only the down-and-local half (propose / fill / ground — acyclic). `reconcile` owns all up-flows (roll-up aggregation and parametric flow-down verification). θ-crosswalk promotion is direction-agnostic. This keeps govern's DAG acyclic by construction. Breadth-vs-depth roll-up strategy is left to neural govern (correctness- neutral).

8.3 The VoI economic model (budget = value-of-information allocator)

Budget is not merely a termination-safety device — it is a value-of-information allocator. Each candidate discharge has:

```
VoI = (marginal value × expected confidence gain) ÷ cost
```

`govern` spends budget in descending VoI order. Termination = top-VoI < cost-floor (nothing left is worth its price). Beyond the existing `gap:budget-exhausted` and `gap:ungrounded`, this introduces two honest terminal gap classes:

- **`gap:trivial`** — could have grounded, had the budget, but it wasn't worth it (low marginal value). Triviality is judged neurally (it is govern's value function).
- **`gap:low-confidence`** / **`gap:confidence-ceiling`** — confidence cannot be raised further for this promise; connects to the plain-attribute terminal state (§4.x). Confidence is judged mixed (σ symbolic + neural).

Seam: triviality → neural, confidence → mixed (σ + neural), cost-floor → symbolic.

Risk collapses into VoI (no new math). Every open promise carries an implicit risk `P(won't ground) × consequence(if it stays a gap)`. `consequence-if-ungrounded` is the VoI value term (the same neural judgment, the inverse of triviality); `P(won't ground)` is the confidence machinery. So risk-weighting is an emergent reading of the existing economics, not a separate term. Mitigation maps to existing ops: reduce/avoid = ground the promise or spawn sub-promises; transfer = re-bind ownership (θ-crosswalk); accept = gate Layer-2 waiver → permanent accepted gap. The risk register is a VIEW over the gap register (sort/filter by consequence); high-consequence open promises surface at gate Layer 2 for explicit, audited human risk-acceptance.

8.4 Gate — two layers

`gate` was the weakest operator in the original design (validity-only). It now carries two layers:

- Layer 1 — symbolic well-formedness. Schema conformance, referential integrity, constraint satisfaction, gap-honesty. Hard: never publish a malformed graph.
- Layer 2 — decision authority (go/no-go). The SE control-gate. Uses the same economic test as termination: go when `top-VoI < cost-floor` (nothing worth doing remains) AND critical-path confidence ≥ acceptance. Symbolic-by-default; escalates to human/neural only on genuine judgment calls (risk-acceptance, waiving permanent gaps).

Provisional roll-ups ARE published with gaps visible — gap-honesty (invariant #5) forbids suppression. A gate "no-go" never hides the graph; it withholds the promotion/acceptance decision, with gaps on display.

---

9. AMS fork manifest

Reuse (fork into openAG):

AMS source	openAG target
`ai/patch.py` (`apply_model_patch`)	Tier-A admit/bind
`walks/`	`WalkEngine`
`lifecycle/` (projectors, renderers, verify)	Tier-A verify + Projection layer
`core/v3_loader.py`, `core/promise.py`, `core/slicer.py`	`ModelGraph` load + `PromiseEngine` + `Slicer`
`pipeline_v3/` (`scan → derive_promises → cluster → link → containerize → verify → publish`)	**`extract`** capability (B1) — the modern 7-stage AST pipeline + the substrate the walk engine runs on
`manifest/scanner.py`, `ts_scanner.py`, `endpoint_extractor`	`SourceScanner` / `PatternDetector`
`feedback/` (append-only journals)	`FeedbackJournals` (+ I/O-entity ownership)
`viewers/`	`Viewer`
Part-14 bundle scheduler	`BudgetGovernor` batching

Drop: only the legacy 10-stage `src/architecture_model/pipeline/` (`observe…emit`). Keep and fork **`pipeline_v3/`** — it is the modern promise-native extractor and openAG's B1 Extract forks it directly (not the legacy linear pipeline).

Forking also fixes two AMS gaps: - AMS A2 "lessons overwritten" → openAG `LessonStore` (append + dedup). - AMS A4 "no rule tuner" → openAG `RuleTuner` (closes the B6 loop).

---

10. Gap register

`gap:open` — none remaining. The three previously-open items (identity θ, govern scheduling, per-kind valence rows) are resolved in §8.1, §8.2, and §4.1 respectively.

Terminal gap taxonomy (honest end-states). A promise that does not reach `verified` ends in exactly one typed gap:

- `gap:ungrounded` — no evidence path exists (the base honest failure).
- `gap:budget-exhausted` — ran out of budget before grounding (§8.2).
- `gap:trivial` — could ground, had budget, not worth it; low marginal value (§8.3; judged neurally).
- `gap:low-confidence` / `gap:confidence-ceiling` — confidence cannot be raised further; connects to the plain-attribute terminal (§15.x).
- `gap:integration-mismatch` — a dynamic interface/behavior promise that failed to ground against the runtime oracle (§16). Not a new mechanism: it is simply an ungrounded dynamic promise.

Integration is NO LONGER deferred — it folds into interface promises + the runtime oracle (§16).

`gap:deferred` (v2 / arch-agent): - A6 Lifecycle axis (expander + `supersedes`/`migrates-to` promises) — but note (§17) lifecycle is a diachronic reading of bind's θ-crosswalk generation history, not a round stage. - Trade-space / decision-management / MOE Pareto analysis — a diachronic meta-ROUND (§17), not a round stage. - Behavior flavors + operational modes. - Federated multi-package growth. - N2 compiler / trainer / distiller + `CAP:mdd` neural backend.

---

11. Consistency analysis (10 checks — all pass)

1. Every L2 sub-capability is realized by ≥1 component. ✓
2. No orphan components (each `realizes`-back to a capability). ✓
3. Every component is assigned to a layer. ✓
4. Every behavior traces to ≥1 component. ✓
5. Every interface has ≥1 exposer and ≥1 consumer. ✓
6. Every actor is connected (consumes/triggers). ✓
7. Total-registry holds — A6 has no expander but no v1 kind owes A6. ✓
8. Gap taxonomy fully sourced from the 3 verification classes. ✓
9. The 4 solutions + N2 all trace to concrete entities/constraints. ✓
10. Attribute algebra needs zero new components (3 loci on existing reconcile/expander machinery). ✓

---

12. Out of scope (this doc)

Implementation plan is deferred. This document is the design only. AMS stays frozen as the production extractor until openAG reaches parity.

---

13. Open-set model + I/O ownership (gen-0 framing)

The entity set is open, not final. The ledger in §5 is Generation 0 — the seed/bootstrap. By openAG's own construction (invariant #5, gap-honesty) the model grows by discharge; declaring it "final" would be dishonest. Growth frontier: the L3 fan-out (§5 note), the `gap:deferred` v2 items (§10), meta-closure (openAG models itself), and the per-discharge `Promise`/`Evidence`/I/O nodes.

All I/O is first-class and owned. Every artifact, journal/log, WorkOrder, Job, Proposal, Note, Lesson, Generation, and Decision is a modeled entity with `owner = <producing entity>`, an inbound `produces` (A5) edge, and hierarchical ownership = the producer's A1↑ **`contains`** chain up to the System. This reuses the lifecycle layer's existing path ownership (`owned_paths` / stale invalidation) and makes "who owns this output?" answerable at any level. Full enumeration in the entity-model companion doc.

---

14. Companion documents

- **`2026-10-04-openAG-entity-model.md`** — gen-0 entity model: full enumeration, non-uniform L3+ functional hierarchy, component/behavior/ interface/actor/layer/constraint attribute rows (anchored to v3 `PromiseSpec` + `axis`/`valence`), the I/O-entity family with ownership, the growth frontier, and the schema-`$defs` reconciliation appendix.
- **`2026-10-04-openAG-pipelines.md`** — B-ROUND + B1–B7 as ordered pipelines, the comparison matrix vs. the 5 existing AMS pipelines (legacy 10-stage, v3 7-stage, walk 6-stage, lifecycle substrate, CAP-DF ceremony), and the operator↔walk↔`model.*` crosswalk.

---

15. B0 — stakeholder-needs elicitation (the front-of-Vee)

B0 seeds System intent + actors + needs before B1 Extract — turning openAG from a pure sustain/reverse-engineering engine into one that can conceive a system from conversation. Six locks:

- B0-1 Output = a seed graph of promised entities. System root + Actor nodes + top-level Capability nodes, each born with OPEN promises from the valence floor, none grounded.
- B0-2 Mechanism = thin ROUND-based coarse seeding. The high-level goal is robust and cheap to acquire ("hard to miss" from README/code, or just ask). Because the goal is coarse and order-insensitive, non-confluence does not bite; fine detail is deferred to confluent ROUND ("as rounds run, things become clear and concrete").
- B0-3 Fact source = artifacts-first, dialogue-to-fill-gaps. Read README/docs/code if present; ask the human only for gaps or greenfield- from-scratch. Dialogue is the fallback, not the default.
- B0-4 Termination = the VoI test, human-attention as budget. The coarse seed makes top-VoI drop fast; un-asked plausible needs become `gap:ungrounded`/plain-attributes, exactly like code gaps.
- B0-5 Need = an OPEN promise owned by an Actor. A need is a `requirement`/`goal`/`success_criteria` promise on the System/Capability with `owner = Actor-id` in provenance (the stakeholder). Grounded by tracing to a realizing capability/component. No new entity kind, no new relation — risk-weighting (via VoI) applies directly.
- B0-6 Seam = no hard boundary; one continuous ROUND, oracle fades. "B0" is a label for the human-oracle-active opening, not a separate phase. The human-dialogue oracle is active early (little grounded) and goes quiet as cheaper code-grounding VoI dominates. No freeze needed (coarse seed is robust + refinement is monotone). This unifies the four fact sources (§3.4) as VoI-arbitrated grounding oracles; the human is simply the most expensive one, used first.

---

16. Integration / right-of-Vee

Integration is not a missing capability — it folds into interface (and behavior) promises grounded by the runtime oracle. Six locks:

- I-1 Integration = grounding interface promises. Interfaces are first-class entities born with open promises; no new mechanism.
- I-2 Non-monotonicity dissolved. "A grounds, B grounds, but A∘B fails" does not require retracting A or B — the failure localizes to the interface's own promise, which was open from the start. The lattice never descends; monotonicity holds for free.
- I-3 "A was actually wrong" → A's promise was under-specified; `grow` proposes a tighter promise and the old one is superseded (`supersedes`/`migrates-to`) — a diachronic Generation change (bind θ-crosswalk), not synchronic retraction. Synchronic ROUND stays monotone.
- I-4 Right-of-Vee = the runtime/composition-execution oracle — the fourth fact source (§3.4), VoI-arbitrated. Integration testing = spending budget to ground dynamic promises via runtime.
- I-5 Two promises per interface. A static-consistency promise (declared contracts line up, grounded left-of-Vee by reconcile/model- check) and a dynamic-composition promise (realized A∘B actually runs, grounded right-of-Vee by the runtime oracle). The two sides of the Vee are two promise-classes; the risk register shows each independently.
- I-6 The pattern generalizes: Behaviors also carry a static (specified-consistent) and a dynamic (actually-executes) promise. "Left/right-of-Vee" is a general promise-class split on any entity with a dynamic aspect.

---

17. SE-completeness audit

Verdict: ROUND is a perfect convergent analytical engine (~60% of systems engineering), not the complete systems engineer.

ROUND covers: architecture definition, verification, decomposition (A1), allocation (A2), traceability (A5), configuration management (via bind), and measurement. With B0 (§15) and Integration (§16) it also covers stakeholder-needs/mission-analysis and integration/right-of-Vee, and risk management folds into VoI (§8.3).

The deep structural thesis. Confluence — the very property that makes neural-govern safe (§8.2) — is antithetical to trade-space, which requires non-confluence (divergent exploration of alternatives). ROUND is convergent / monotone / evidence-grounded; full SE is also divergent-then-convergent / non-monotone / requirement-driven.

Therefore the two remaining SE concerns are DIACHRONIC / meta-ROUND, not round stages:

- Trade-space / decision-management = a meta-loop that runs ROUND per candidate architecture and selects a winning Generation. Each inner ROUND stays confluent; the divergence lives in the outer loop that spawns candidates. (Deferred; placed, not detailed.)
- Lifecycle (A6) = a diachronic reading of bind's θ-crosswalk generation history. (Deferred.)

ROUND stays synchronic (one architecture at one timepoint). The across-time layer is the B-behaviors + `GenerationStore`.

---

18. Attribute end-states + reservoir (summary)

Grow proposes above the valence floor into a reservoir (a staging buffer + owner-less parking lot); ground promotes only what verifies. The floor is a declarative well-formedness type-signature, not a schedule (so it does not violate no-hardcoded-policy). Floor fixed, ceiling emergent. A promise resolves to exactly four end-states:

1. verified promise — grounded; in promise accounting.
2. reservoir (open) — owned/collected but not yet grounded.
3. plain attribute — a legitimate terminal non-gap state: non-verifiable content (prose/rationale) is retained but untracked, never in promise accounting. Verifiability gate: grow proposes → ground asks "is there any evidence path?" → yes = promote to promise, no = stays plain attribute.
4. typed-gap — the only one of the four that counts as a gap.

Reservoir→owner matching is the θ-crosswalk by content σ — the same operator/thresholds as module rebinding (§2.3, §8.1). Unifying insight: `reconcile` + σ is the single binding operator for modules, attributes, AND cross-generation identity.